

Proyecto LLM Chat

Plataforma SaaS Multi-Tenant, Arquitectura Laravel 11, Base de Datos MySQL, IA n8n en Docker y Conexión con App Android en Kotlin

Ingeniería en Desarrollo de Software

Universidad Autónoma de Chiapas

6 de octubre de 2026

Presentación Ejecutiva del Proyecto

El **Proyecto LLM Chat** es una solución tecnológica avanzada bajo el modelo **SaaS Multi-Tenant** (Software as a Service) diseñada para la publicación y gestión inteligente de catálogos digitales de productos y precios. Cada empresa registrada posee un espacio aislado (*tenant*) y un código QR único. Los clientes pueden consultar la información escaneando el código QR o desde una aplicación móvil nativa en **Kotlin** que se comunica mediante una API REST en **Laravel** e integra un filtro inteligente respaldado por **n8n en Docker**.

Índice

1. 1. Objetivo General y Flujo de la Plataforma	3
2. 2. Jerarquía de Roles y Control de Acceso (RBAC)	4
3. 3. Modelo de Datos y Diagrama ER de Tablas MySQL	5
4. 4. Automatización e IA con n8n en Docker	6
5. 5. Conexión de la App Móvil Android (Kotlin) con Laravel API	7
5.1. 1. Definición de la API en Kotlin	7
5.2. 2. Extracción del QR en Kotlin	7
6. 6. Resumen de Tecnologías y Conclusión	8

1. 1. Objetivo General y Flujo de la Plataforma

El objetivo general es proveer una infraestructura centralizada en la nube que permita:

1. **Administración General (SaaS Operator):** Gestionar empresas clientes, planes de suscripción, usuarios y auditoría global.
2. **Gestión Empresarial (Tenants):** Permitir a cada empresa dar de alta productos, categorías, establecer precios y obtener su código QR exclusivo.
3. **Consulta de Clientes Móviles:** Permitir que los clientes escaneen el código QR o utilicen una App Android en Kotlin para acceder al catálogo actualizado en tiempo real sin requerir registro obligatorio.

Actor / Capa	Componente	Acción Principal
Operador SaaS	Panel Administrador General	Alta de empresas, planes y logs de auditoría.
Empresas	Panel Multi-Tenant (company_id)	Alta de catálogo, precios y código QR.
Clientes / App	Android App Kotlin / Escáner QR	Lectura de catálogo e IA inteligente.

2. Jerarquía de Roles y Control de Acceso (RBAC)

La seguridad del sistema está respaldada por una jerarquía estricta de tres roles:

Rol	Jerarquía	Reglas de Acceso
admin	Nivel 1 (Máximo)	Acceso total a empresas, usuarios, logs e integraciones globales.
empresa	Nivel 2 (Corporativo)	Gestión exclusiva de los productos y precios vinculados a su propia empresa. Datos aislados de otros tenants.
cliente	Nivel 3 (Público/Mobile)	Registro público predeterminado desde la App Android. Acceso a lectura del catálogo y consultas al asistente LLM.

Seguridad en Registro Público

Cualquier registro de usuario realizado desde la interfaz pública o desde la App Android asigna **exclusiva y obligatoriamente** el rol `cliente`, evitando elevaciones de privilegio no autorizadas.

3. Modelo de Datos y Diagrama ER de Tablas MySQL

La base de datos relacional se gestiona en **MySQL 8.0** con aislamiento por `company_id`:

```
-- 1. Tabla de Empresas (Tenants)
CREATE TABLE companies (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  phone VARCHAR(50) NULL,
  plan ENUM('Basico', 'Plus', 'Premium') DEFAULT 'Basico',
  status ENUM('Activo', 'Inactivo') DEFAULT 'Activo',
  created_at TIMESTAMP, updated_at TIMESTAMP
);

-- 2. Tabla de Usuarios vinculados a Tenant
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'empresa', 'cliente') DEFAULT 'cliente',
  company_id BIGINT NULL,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE SET
    NULL
);

-- 3. Tabla de Productos por Empresa
CREATE TABLE products (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  company_id BIGINT NOT NULL,
  category_id BIGINT NULL,
  name VARCHAR(255) NOT NULL,
  description TEXT NULL,
  price DECIMAL(10,2) NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE
);

-- 4. Bitacora Inmutable de Auditoria
CREATE TABLE audit_logs (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NULL,
  action VARCHAR(255) NOT NULL,
  ip_address VARCHAR(45) NULL,
  details TEXT NULL,
  created_at TIMESTAMP, updated_at TIMESTAMP
);
```

Listing 1: Esquema SQL de las tablas principales

4. 4. Automatización e IA con n8n en Docker

Para procesar búsquedas semánticas e inteligentes sobre los catálogos de los clientes, se utiliza un motor **n8n** ejecutado en un contenedor **Docker**.

```
version: '3.8'
services:
  app:
    build: .
    ports: ["8000:8000"]
  n8n:
    image: docker.n8n.io/n8nio/n8n:latest
    ports: ["5678:5678"]
    environment:
      - N8N_HOST=n8n
  db:
    image: mysql:8.0
    ports: ["3306:3306"]
```

Listing 2: Configuración docker-compose.yml

Flujo del Filtro Inteligente: 1. La App Android envía una consulta a `POST /api/v1/n8n/smart-filter`
2. Laravel reenvía los productos del tenant al webhook de n8n en Docker (`http://n8n:5678/webhook/sm`)
3. n8n procesa la consulta con nodos de IA y devuelve los productos más relevantes.

5. 5. Conexión de la App Móvil Android (Kotlin) con Laravel API

La aplicación nativa Android escrita en **Kotlin** se conecta a las APIs REST de Laravel utilizando **Retrofit** y **Corrutinas**.

5.1. 1. Definición de la API en Kotlin

```
package com.llmchat.app.data.api

import retrofit2.Response
import retrofit2.http.GET
import retrofit2.http.Path
import retrofit2.http.POST
import retrofit2.http.Body

interface CatalogApiService {
    // Escaneo de QR o consulta de catalogo por Slug
    @GET("api/v1/catalogo/{slug}")
    suspend fun getCatalogBySlug(
        @Path("slug") slug: String
    ): Response<CatalogResponse>

    // Filtro Inteligente respaldado por n8n
    @POST("api/v1/n8n/smart-filter")
    suspend fun filterCatalog(
        @Body request: SmartFilterRequest
    ): Response<SmartFilterResponse>
}
```

Listing 3: CatalogApiService.kt en Kotlin

5.2. 2. Extracción del QR en Kotlin

```
object QrParserUtil {
    fun extractSlugFromQr(qrUrl: String): String {
        return qrUrl.trim().substringAfterLast("/q/").trimEnd('/')
    }
}
```

Listing 4: QrParserUtil.kt

6. 6. Resumen de Tecnologías y Conclusión

Tecnología	Componente	Función Principal
Laravel 11	Backend REST API & Blade	Servidor principal, APIs JSON, middle-ware de roles e interfaz Blade.
MySQL 8.0	Base de Datos	Almacenamiento de empresas, productos, categorías y auditoría.
n8n (Docker)	Motor de Automatiza-ción	Filtros inteligentes e inteligencia artificial en contenedor.
Kotlin (Android)	App Nativa Móvil	Lectura de código QR y consumo de APIs REST.
Tailwind CSS	Diseño Web	Interfaz moderna con tema oscuro y con-tadores de sesión en tiempo real.

Conclusión

La plataforma **LLM Chat** combina la solidez de Laravel 11 y MySQL con la flexibilidad de n8n en Docker y la potencia de Kotlin en Android, ofreciendo un SaaS multi-tenant completo, seguro y escalable.